Project Proposal: Agentic AI Applications Course

Team Number: 08

Team Member Name(s): Jadesh V, Lalith Tejas M Hiremath, Manohar Reddy M.

Proposed Project Title: Roam-Ind: A Real-Time Intelligent Travel Assistance Framework

1. Problem Statement & Use Case

- **Real-World Problem:** Travel in India is plagued by dynamic pricing, lack of transparency, safety concerns, and unpredictable traffic/weather that static apps cannot handle.
- **Target User:** Foreign and domestic travelers seeking adaptive, real-time guidance.
- **Current State & Pain Points:** Existing tools provide fragmented services and static itineraries that fail to adapt to real-time events like surge pricing or medical emergencies.
- **Why Agentic AI?** A simple LLM cannot perform continuous monitoring, multi-step tool coordination, or autonomous re-planning based on live data feeds.

2. Justification of Agentic Design

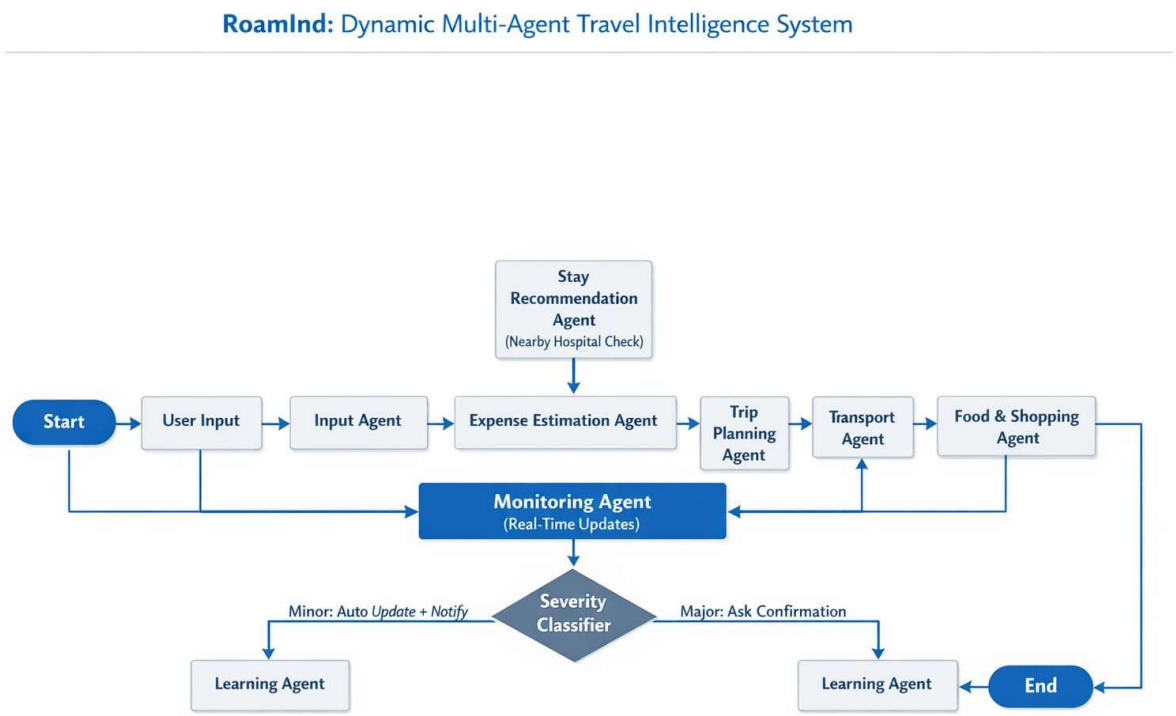
- **Core Capabilities:** The framework relies on **tool use** (fetching live fares), **planning** (itinerary decomposition), and **reflection** (a learning agent that refines logic based on user behavior).
- **Failure of Single LLM Call:** A standard chain would provide a one-time static response and would be unable to classify change severity or monitor conditions during the trip.
- **Expected Behavior:** The agent must detect inconsistent data (e.g., a hotel closure), reflect on the impact, and autonomously propose a major or minor update.

3. High-Level Solution Overview

RoamInd uses a Multi-Agent Architecture where specialized agents handle distinct travel domains.

- **Main Agents:** Input Agent, Expense Agent, Stay Recommendation Agent, Trip Planning Agent, Transport Agent, and a Monitoring Agent.
- **Agentic Patterns:**
  - **Reflection:** A learning agent updates ranking logic.
  - **Planning:** Hierarchical plan-and-execute for itineraries.
  - **Multi-Agent:** Coordinator/Supervisor model to synthesize agent outputs.
- **High-Level Flow:** User Input → Planner decomposes → Specialized Agents execute (Stay, Food, Transport) → Monitoring Agent watches for changes → Severity Classifier updates.

Here is an example diagram:



4. Proposed Technology Stack

- **LLM & Framework:** GPT-4o or Gemini Pro utilizing **LangGraph** for stateful, cyclic workflows.
- **Tools:** Tavily for search, Google Maps API for traffic, and custom APIs for Indian ride-hailing and medical facility databases.
- **Memory: ChromaDB** for long-term user preferences and travel history.
- **Frontend & Eval: Streamlit** for the UI and **LangSmith** for tracking agentic traces and tool-call accuracy.

5. Success Metrics & Evaluation Plan

- **Quantitative:** Task success rate in re-planning, end-to-end latency, and tool-call accuracy.
- **Qualitative:** Rubric scores for itinerary coherence and user satisfaction surveys with a test group of 10 people.
- **Failure Modes:** Monitoring for infinite loops in re-planning or cost spikes from frequent API polling.

6. Risks, Challenges & Mitigation Plan

- **Hallucinations:** Mitigated by reflection loops and mandatory source citation for pricing.
- **Infinite Loops:** Addressed via strict recursion limits and max-iteration caps.
- **Tool Failures:** Error-handling nodes and fallback prompts in the agent graph.

7. Timeline & Milestones

- **Mar 25:** Single-agent prototype with expense estimation and tool use.
- **Apr 10:** Multi-agent design and early evaluation of the Monitoring Agent.
- **Apr 22:** Multi-agent MVP with a functional dashboard and severity classification.

- **May 05:** Final polished product with learning-driven recommendations.